

Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Akanksha Kumari

Roll No: 2501010076

Email: 2501010076@krmu.edu.in

Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def compare_search_algorithms(arr, target):
    linear_idx=-1
    linear_comps=0
    for i in range(len(arr)):
        linear_comps+=1
        if arr[i]==target:
            linear_idx=i
            break
    binary_idx=-1
    binary_comps=0
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        binary_comps+=1
        if arr[mid]==target:
            binary_idx=mid
            high=mid-1
        elif arr[mid]<target:
            low=mid+1
        else:
            high=mid-1
    if linear_comps<binary_comps:
        better="Better Algorithm: Linear Search"
    elif binary_comps<linear_comps:
        better="Better Algorithm: Binary Search"
    else:
        better="Better Algorithm: Both Equal"
    return[
        "Search Comparison Report",
        "Linear Search",
        f"Index: {linear_idx}",
        f"Comparisons: {linear_comps}",
        "Binary Search",
        f"Index: {binary_idx}",
        f"Comparisons: {binary_comps}",
        better,
    ]
```

Q2. 1.1 Recursive and Iterative Computation Analyzer (10 marks)

```
def analyze_recursive_iterative(n):
    rec_fact_calls=0
    def fact_rec(x):
        nonlocal rec_fact_calls
        rec_fact_calls+=1
        if x<=0:
            return 1
        return x*fact_rec(x-1)
    rec_fact_val=fact_rec(n)
    iter_fact_val=1
    iter_fact_count=n
    for i in range(1,n+1):
        iter_fact_val*=i
    rec_fib_calls=0
    def fib_rec(x):
        nonlocal rec_fib_calls
        rec_fib_calls+=1
        if x<=0:
            return 0
        if x==1:
            return 1
        return fib_rec(x-1)+fib_rec(x-2)
```

```

rec_fib_val=fib_rec(n)
if n==0:
    iter_fib_val=0
elif n==1:
    iter_fib_val=1
else:
    a,b=0,1
    for _ in range(2,n+1):
        a,b=b,a+b
    iter_fib_val=b
iter_fib_count=n
return[
    "Computation Analysis Report",
    f"Recursive Factorial: {rec_fact_val}",
    f"Iterative Factorial: {iter_fact_val}",
    f"Recursive Fibonacci: {rec_fib_val}",
    f"Iterative Fibonacci: {iter_fib_val}",
    "Operation Count Comparison",
    f"Recursive Factorial Count: {rec_fact_calls}",
    f"Iterative Factorial Count: {iter_fact_count}",
    f"Recursive Fibonacci Count: {rec_fib_calls}",
    f"Iterative Fibonacci Count: {iter_fib_count}",
]

```

Q3. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```

def compare_bubble_insertion(random_data, sorted_data, reverse_data):
    def bubble_sort(arr):
        a=list(arr)
        n=len(a)
        comps=0
        swaps=0
        for i in range(n):
            swapped=False
            for j in range(0,n-i-1):
                comps+=1
                if a[j]>a[j+1]:
                    a[j],a[j+1]=a[j+1],a[j]
                    swaps+=1
                    swapped=True
            if not swapped:
                break
        return a,comps,swaps
    def insertion_sort(arr):
        a=list(arr)
        n=len(a)
        comps=0
        shifts=0
        for i in range(1,n):
            key=a[i]
            j=i-1
            while j>=0:
                comps+=1
                if a[j]>key:
                    a[j+1]=a[j]
                    shifts+=1
                    j-=1
                else:
                    break
            a[j+1]=key
        return a,comps,shifts
    datasets=[
        ("Random Dataset", random_data),
        ("Sorted Dataset", sorted_data),
        ("Reverse Dataset", reverse_data),
    ]
    report=["Sorting Performance Report"]
    for name,data in datasets:
        b_sorted,b_comps,b_swaps=bubble_sort(data)
        i_sorted,i_comps,i_shifts=insertion_sort(data)
        if b_comps<i_comps:
            better="Better Algorithm: Bubble Sort"
        elif i_comps<b_comps:
            better="Better Algorithm: Insertion Sort"
        else:
            better="Better Algorithm: Both Equal"
        b_str=" ".join(map(str,b_sorted))

```

```
i_str=" ".join(map(str,i_sorted))
report.extend([
    name,
    f"Bubble Sorted: {b_str}",
    f"Bubble Comparisons: {b_comps}",
    f"Bubble Swaps: {b_swaps}",
    f"Insertion Sorted: {i_str}",
    f"Insertion Comparisons: {i_comps}",
    f"Insertion Shifts: {i_shifts}",
    better,
])
# your code goes here
return report
```